

Breno Capraro de Souza, Cauã Ribas Devitte, Nilson Andrade Neto

Universidade do Vale do Itajaí - Univali
Escola Politécnica
Ciência da Computação
{breno.net, cauaribas, nilson.neto} @edu.univali.br

Processamento de Imagens
Trabalho M1 – Operações Pontuais e Filtragem Espacial
Felipe Viel

14/04/2025

1. **Introdução**
2. **Explicação e contexto da aplicação para compreensão do problema tratado pela solução**
3. **Desenvolvimento**
 - 3.1. Dataset utilizado
4. **Códigos importantes de implementação.**
 - 4.1. Ruído Sal e Pimenta
 - 4.2. Equalização de Histograma
 - 4.3. Correção de Gama
 - 4.4. Convolução
 - 4.5. Filtro Laplaciano
 - 4.6. Operador de Sobel
 - 4.7. Filtro de Alta Frequência (High-Boost Filtering)
 - 4.8. Filtro Gaussiano
 - 4.9. Filtro Mediana
5. **Resultados obtidos com a implementação**
6. **Análise e discussão sobre os resultados finais (inclusive de problemas)**
7. **Conclusão**
8. **Referências**

1. Introdução

O **processamento digital de imagens** é uma das áreas mais impactantes da Ciência da Computação aplicada à saúde, tendo papel fundamental no apoio a diagnósticos por imagem. Com o uso crescente de exames digitais em clínicas e hospitais, torna-se cada vez mais necessário aplicar técnicas que aprimorem a legibilidade e a qualidade visual dessas imagens, possibilitando uma análise mais segura e eficiente por parte dos profissionais da saúde.

Neste projeto, o foco está no **PathMNIST**, subconjunto do repositório **MedMNIST v2**, composto por imagens histopatológicas do cólon humano, rotuladas de acordo com diferentes classes de tecido. Tais imagens são particularmente desafiadoras por apresentarem características como **texturas complexas**, **baixo contraste** entre regiões, além de estarem suscetíveis à presença de **ruídos** e **bordas mal definidas**.

Com base nesse cenário, o trabalho propõe a aplicação de uma sequência estruturada de **operações pontuais e técnicas de filtragem espacial**, com o objetivo de restaurar e realçar as imagens, favorecendo a extração de informações clínicas relevantes. A proposta busca evidenciar como o uso coordenado dessas técnicas pode melhorar significativamente a interpretação visual em aplicações voltadas à patologia digital.

2. Explicação e contexto da aplicação para compreensão do problema tratado pela solução

Imagens médicas desempenham um papel essencial na área da saúde, auxiliando na análise, diagnóstico e acompanhamento de condições clínicas. No entanto, por serem capturadas por dispositivos eletrônicos, essas imagens estão sujeitas a diversas formas de distorção que afetam sua qualidade visual e, consequentemente, a precisão da interpretação médica.

Entre os principais desafios enfrentados no processamento dessas imagens, destacam-se:

- **Baixo Contraste:** Tecidos e estruturas anatômicas podem apresentar intensidades muito semelhantes, dificultando sua diferenciação visual.
- **Presença de Ruídos do Tipo Sal e Pimenta:** Frequentes em sensores digitais, esses ruídos inserem pixels com valores extremos, comprometendo a nitidez e obscurecendo detalhes importantes.
- **Esmaecimento de Bordas:** A suavização natural de algumas regiões pode dificultar a delimitação precisa de contornos e áreas de interesse clínico.
- **Perda de Detalhes com Filtragens Excessivas:** Técnicas de suavização mal calibradas podem eliminar ruídos, mas também remover informações cruciais da imagem.

Diante desses desafios, a solução proposta neste trabalho consiste na aplicação de um conjunto de técnicas de **operações pontuais** e **filtragem espacial**, implementadas de forma sequencial e controlada, para restaurar e aprimorar a qualidade das imagens.

A solução proposta neste trabalho envolve a aplicação de operações pontuais como **equalização de histograma** e **correção gama** para melhorar o contraste, e técnicas de filtragem espacial como **filtro gaussiano, mediana, laplaciano, sobel e high-boost filtering** para suavização e aguçamento da imagem.

3. Desenvolvimento

3.1. Dataset Utilizado:

O conjunto de dados utilizado neste projeto foi o **PathMNIST**, parte da coleção **MedMNIST v2**, voltado à área da saúde. Este dataset consiste em imagens médicas patológicas coloridas, previamente normalizadas para uma resolução de **128x128 pixels**. Para facilitar o processamento e se adequar aos objetivos do trabalho, as imagens foram convertidas para escala de cinza, permitindo a aplicação direta de operações pontuais e filtragem espacial.

3.2. Etapas do Processamento

A seguir, descrevem-se as etapas implementadas no pipeline de processamento, com ênfase em técnicas manuais e implementações próprias, conforme as exigências do projeto:

3.2.1. Adição de Ruído (Sal e Pimenta)

Inicialmente, foi introduzido ruído do tipo "**Sal e Pimenta**" nas imagens. Este ruído simula interferências comuns na aquisição de imagens médicas, permitindo avaliar a eficácia das técnicas de suavização e restauração subsequentes. A aplicação foi feita de forma controlada, variando a intensidade do ruído para analisar os efeitos sobre a qualidade da imagem.

3.2.2. Operações Pontuais: Ajuste de Contraste

Para mitigar os efeitos de baixo contraste — um problema recorrente em imagens médicas, duas abordagens distintas foram implementadas:

- **Equalização de Histograma:** redistribuição dos níveis de cinza da imagem para ampliar a gama tonal e melhorar a distinção entre diferentes regiões da imagem.

- **Correção Gama:** ajuste da intensidade dos pixels utilizando uma função de potência (gama), o que permite enfatizar detalhes de baixa ou alta intensidade, dependendo do valor utilizado.

Ambas as técnicas foram testadas individualmente, e os resultados foram comparados visualmente para seleccionar a que apresentou melhor desempenho em termos de contraste e realce de detalhes relevantes.

3.2.3. Filtragem Espacial para Suavização

A etapa de suavização teve como objetivo a **redução de ruído**, preservando ao máximo as estruturas anatômicas importantes. Foram implementadas duas abordagens clássicas:

- **Filtro Gaussiano:** suaviza a imagem por meio da convolução com uma máscara gerada a partir da distribuição normal, atenuando ruídos aleatórios.
- **Filtro Mediana:** mais eficaz para ruídos do tipo "sal e pimenta", substitui cada pixel pelo valor mediano de sua vizinhança, mantendo as bordas preservadas.

A escolha do filtro aplicado foi baseada na análise da imagem com ruído introduzido, visando obter o melhor equilíbrio entre remoção de ruído e preservação de bordas.

3.2.4. Filtragem Espacial para Realce (Aguçamento)

Com o objetivo de **realçar bordas** e estruturas de interesse, foram aplicadas técnicas de filtragem espacial de alta frequência, utilizando diferentes operadores:

- **Filtro Laplaciano:** aplicado para destacar regiões de transição abrupta de intensidade, facilitando a visualização de contornos e detalhes finos.
- **Operador de Sobel:** detecta gradientes de intensidade na imagem, acentuando bordas horizontais e verticais.
- **High-Boost Filtering:** combina a imagem original com sua versão suavizada, amplificando as componentes de alta frequência (detalhes), sem perda da estrutura geral.

4. Códigos Importantes da Implementação

4.1. Ruído Sal e Pimenta:

```
1  # Sal e Pimenta
2  def add_salt_and_pepper(image, prob=0.10):
3      output = np.copy(image)
4      h, w = image.shape
5
6      random_matrix = np.random.rand(h, w)
7      pepper_mask = random_matrix < (prob / 2)
8      salt_mask = random_matrix > 1 - (prob / 2)
9
10     output[pepper_mask] = 0
11     output[salt_mask] = 255
12
13     return output
```

Esta função tem como objetivo **adicionar ruído do tipo "Sal e Pimenta"** a uma imagem em escala de cinza. Esse tipo de ruído simula distorções comuns em sensores digitais, onde alguns pixels assumem valores extremos (preto ou branco) de maneira aleatória.

4.2. Equalização de histograma:

```
1  # Equalização de histograma
2  def histogram_equalization(img):
3      height, width = img.shape
4      total_pixels = height * width
5      hist = [0] * 256
6      for i in range(height):
7          for j in range(width):
8              hist[int(img[i, j])] += 1
9      cdf = [0] * 256
10     cdf[0] = hist[0]
11     for i in range(1, 256):
12         cdf[i] = cdf[i-1] + hist[i]
13     cdf_normalized = [int(round((cdf[i] / total_pixels) * 255)) for i in range(256)]
14     equalized_img = np.zeros_like(img, dtype=np.uint8)
15     for i in range(height):
16         for j in range(width):
17             equalized_img[i, j] = cdf_normalized[int(img[i, j])]
18     return equalized_img
```

Esta função tem como finalidade aplicar a técnica de **equalização de histograma** em uma imagem em tons de cinza. Essa técnica é amplamente utilizada no processamento de imagens para **melhorar o contraste**, redistribuindo os níveis de intensidade de modo a explorar toda a faixa possível (de 0 a 255).

Tem como objetivo transformar a distribuição de intensidade dos pixels para que fique o mais uniforme possível, aumentando a separação entre tons semelhantes e tornando detalhes mais perceptíveis, especialmente em imagens com faixas de intensidade restritas. A imagem de entrada em escala de cinza, representada como um array 2D do NumPy com valores entre 0 e 255. Captura-se a dimensão da imagem e calcula-se o total de pixels, necessário para normalizar os valores posteriormente. Cria-se um vetor de 256 posições (um para cada nível de intensidade de 0 a 255). A cada pixel percorrido, incrementa-se o valor correspondente ao seu nível de cinza no histograma. A **CDF (Cumulative Distribution Function)** é construída a partir do histograma, somando os valores cumulativos. Ela representa a frequência acumulada de cada nível de intensidade. A CDF é normalizada para se ajustar ao intervalo de intensidade da imagem (0 a 255), transformando os valores em novos níveis de cinza. Cada pixel da imagem original é substituído pelo novo valor correspondente da CDF normalizada.

Esse mapeamento resulta em uma imagem com melhor distribuição de tons. A função retorna a nova imagem processada, com contraste aprimorado.

3.3. Correção de Gama:

```
1  # Correção gama
2  def gamma_correction(img, gamma=1.0):
3      height, width = img.shape
4      corrected_img = np.zeros_like(img, dtype=np.uint
5      8)
6      for i in range(height):
7          for j in range(width):
8              normalized = img[i, j] / 255.0
9              corrected = math.pow(normalized, 1/gamm
10             a)
11              corrected_img[i, j] = int(round(correcte
12             d * 255))
13      return corrected_img
```

A função aplica a técnica de **correção gama** a uma imagem em escala de cinza. Essa operação pontual tem como objetivo ajustar o brilho e a distribuição tonal da imagem, sendo especialmente útil para **melhorar a visibilidade de detalhes em regiões muito escuras ou muito claras**, conforme o valor do parâmetro gama. Imagem de entrada (grayscale), representada como uma matriz NumPy 2D com valores entre 0 e 255. O valor do fator gama por padrão, é 1.0 (default value), mas ao variar o valor é possível clarear (**gamma < 1**) ou escurecer (gamma > 1) a imagem. O processo consiste em criar uma nova imagem com as mesmas dimensões da original, iniciando com todos os pixels em zero. Para cada pixel da imagem, o valor é inicialmente normalizado para o intervalo [0, 1] por meio da divisão por 255. Em seguida, aplica-se a transformação gama utilizando a função $f(x) = x^{1/\gamma}$, onde γ é o parâmetro definido pelo usuário. Quando γ é menor que 1, a imagem resultante tende a ficar mais clara; por outro lado, quando γ é maior que 1, a imagem torna-se mais escura. Após essa transformação, o valor corrigido é reescalado de volta para o intervalo [0, 255] e convertido para o formato inteiro **uint8**, de modo a compor corretamente a imagem final.

3.4. Convolução:

```
1  # Função de convolução
2  def convolve2d(image, kernel):
3      k_h, k_w = kernel.shape
4      pad_h = k_h // 2
5      pad_w = k_w // 2
6      padded = add_padding(image, pad_h, pad_w, constant_value=0)
7      output = np.zeros_like(image, dtype=np.float64)
8      for i in range(image.shape[0]):
9          for j in range(image.shape[1]):
10             region = padded[i:i+k_h, j:j+k_w]
11             output[i, j] = np.sum(region * kernel)
12     return output
```

Esta função realiza a **operação de convolução bidimensional** entre uma imagem em escala de cinza e um kernel fornecido, sendo essa uma das técnicas fundamentais no processamento de imagens. A convolução é utilizada para diversas finalidades, como suavização, detecção de bordas, realce de detalhes e filtragem em geral.

Inicialmente, a função obtém as dimensões do kernel e calcula o deslocamento necessário para manter o alinhamento centralizado durante a operação. Com base nesses valores, é aplicado um **preenchimento (padding)** à imagem original, adicionando bordas de zeros ao redor da imagem, de forma que a saída resultante tenha as mesmas dimensões da entrada. O preenchimento é realizado pela função auxiliar **add_padding**, utilizando como valor constante o zero, o que equivale a uma borda preta.

Após o preenchimento, inicia-se o processo de convolução. Para cada posição da imagem original, uma região do mesmo tamanho do kernel é recortada da imagem com padding. Essa região é então multiplicada ponto a ponto pelo kernel, e os valores resultantes são somados. O valor final é atribuído ao pixel correspondente da imagem de saída.

A imagem resultante é construída iterativamente e armazenada em um array output, com valores em ponto flutuante (**float64**), o que permite maior precisão no cálculo. Ao final, a função retorna essa imagem convoluída, que pode representar, dependendo do kernel utilizado, uma imagem suavizada, realçada ou com bordas destacadas.

3.5. Filtro Laplaciano:

```
1  # Definição dos Kernels Laplacianos
2  laplacian_kernel = np.array([[ 0, -1,  0],
3                               [-1,  4, -1],
4                               [ 0, -1,  0]])
5  averaging_kernel = np.ones((3, 3)) / 9.0
6
7  # Filtro Laplaciano
8  # aplicando um alpha para o filtro não ser tão agressivo
9  def laplacian_filter(image, alpha = 0.5):
10     laplacian_img = convolve2d(image, laplacian_kernel)
11     sharpened = image - alpha * laplacian_img
12     sharpened = np.clip(sharpened, 0, 255)
13     return sharpened
14
15
```

Esta função aplica um filtro de realce baseado no operador **Laplaciano**, amplamente utilizado para **destacar bordas e transições abruptas de intensidade** em imagens. O núcleo (kernel) Laplaciano utilizado é uma matriz 3x3.

Esse kernel calcula a segunda derivada da intensidade dos pixels, realçando regiões onde há mudanças bruscas como contornos e arestas. Para evitar que o efeito do filtro seja excessivamente agressivo, é introduzido o parâmetro **alpha**, que controla a intensidade do realce aplicado.

O processo inicia-se com a convolução da imagem original com o kernel Laplaciano, por meio da função **convolve2d**, o que resulta em uma imagem contendo as regiões com maior variação de intensidade. Em seguida, essa imagem é **subtraída** da original, ponderada pelo fator **alpha**, com o intuito de ressaltar os detalhes sem comprometer a estrutura geral da imagem. Essa abordagem é uma forma de **aguçamento por realce negativo**, onde a imagem realçada é obtida por:

$$I_{\text{sharpened}} = I_{\text{original}} - \alpha \cdot I_{\text{laplaciano}}$$

O resultado final é passado por uma operação de **clipping**, limitando os valores dos pixels ao intervalo [0, 255], garantindo que permaneçam dentro dos limites válidos para imagens em escala de cinza. A imagem retornada apresenta detalhes mais nítidos e bordas mais definidas, o que é especialmente útil em imagens médicas com **bordas esmaecidas**, como as do dataset **PathMNIST**.

3.6 Sobel:

```
1 def sobel_filter(img):
2     kernel_x = np.array([
3         [-1, 0, 1],
4         [-2, 0, 2],
5         [-1, 0, 1]
6     ])
7     kernel_y = np.array([
8         [-1, -2, -1],
9         [ 0,  0,  0],
10        [ 1,  2,  1]
11    ])
12
13    img_height, img_width = img.shape
14
15    padded_img = np.pad(img, pad_width=1, mode='edge')
16    output_img = np.zeros((img_height, img_width), dtype=float)
17
18    for i in range(1, img_height + 1):
19        for j in range(1, img_width + 1):
20            gx = np.sum(padded_img[i-1:i+2, j-1:j+2] * kernel_x)
21            gy = np.sum(padded_img[i-1:i+2, j-1:j+2] * kernel_y)
22            output_img[i-1, j-1] = abs(gx) + abs(gy)
23
24    max_val = output_img.max()
25    if max_val > 0:
26        output_img = (output_img / max_val) * 255
27    return output_img.astype(np.uint8)
```

A função **sobel_filter** aplica o **filtro de Sobel**, uma técnica clássica de **detecção de bordas** baseada na aproximação do gradiente da intensidade da imagem. Esse filtro é amplamente utilizado por sua capacidade de destacar contornos de forma eficiente, sendo útil para realçar estruturas em imagens médicas, como as do dataset **PathMNIST**, onde as bordas celulares podem estar suavizadas.

O algoritmo utiliza dois kernels distintos: **kernel_x**, que detecta variações horizontais (bordas verticais), e **kernel_y**, responsável por detectar variações verticais (bordas horizontais). Ambos os kernels são máscaras convolucionais 3x3 com pesos que enfatizam as diferenças locais de intensidade em suas respectivas direções.

A imagem é primeiramente **preenchida com bordas replicadas** (padding com **mode='edge'**) para garantir que a convolução possa ser aplicada a todos os pixels, inclusive nos limites. Em seguida, para cada pixel da imagem, é extraída uma janela 3x3 da imagem com padding. Essa janela é multiplicada elemento a elemento (produto de Hadamard) pelos kernels **kernel_x** e **kernel_y**, cujos resultados são somados para obter os gradientes G_x e G_y nas direções horizontal e vertical, respectivamente. A magnitude do gradiente total é então estimada pela soma dos módulos de G_x e G_y , o que fornece uma aproximação eficiente da intensidade da borda naquele ponto.

Após o cálculo de todos os gradientes, a imagem resultante é **normalizada** para o intervalo $[0, 255]$, utilizando o valor máximo presente na imagem de saída. Por fim, os dados são convertidos para o tipo **uint8**, gerando uma imagem em tons de cinza com as bordas destacadas.

3.7. Filtro de Alta Frequência (High-Boost Filtering)

```
1 # Filtro High-Boost
2 def highboost_filter(image, A=3.0):
3     blurred = convolve2d(image, averaging_kernel)
4     mask = image - blurred
5     highboost = image + (A - 1) * mask
6     highboost = np.clip(highboost, 0, 255)
7     return highboost
8
```

Esta função implementa a técnica de **High-Boost Filtering**, um método de realce que combina a imagem original com uma versão suavizada (borrada) para **reforçar detalhes e bordas** sem perder a estrutura geral. Essa técnica é particularmente útil em imagens médicas com regiões homogêneas e detalhes sutis, como as histopatológicas do dataset **PathMNIST**, onde é essencial destacar pequenas variações estruturais.

O processo inicia com a aplicação de uma **suavização** sobre a imagem original por meio da convolução com um kernel de média 3x3 (**averaging_kernel**), que calcula a média dos pixels vizinhos, resultando em uma imagem borrada. Em seguida, é construída uma **máscara de alta frequência** ao subtrair a imagem borrada da original. Essa máscara evidencia os detalhes da imagem — como contornos e texturas — que foram suavizados no processo de desfoque. A etapa seguinte consiste em **recompôr a imagem**, somando à original um fator escalado da máscara. Essa recomposição é feita pela equação:

$$I_{highboost} = I_{original} + (A - 1) \cdot (I_{original} - I_{borrada})$$

Onde A é um parâmetro de amplificação. Quando $A = 1$, a técnica equivale ao realce por subtração simples (imagem menos borrada); valores maiores de A intensificam o reforço dos detalhes.

Por fim, os valores da imagem resultante são limitados ao intervalo $[0, 255]$ por meio da função **np.clip**, garantindo que todos os pixels estejam em um intervalo válido para exibição em escala de cinza. O resultado final é uma imagem com **textura e bordas mais evidentes**, sem comprometer sua estrutura global, sendo ideal como etapa final de realce em um pipeline de pré-processamento de imagens médicas.

3.8. Filtro Gaussiano

```
1  # Kernel Gaussiano
2  def gaussian_kernel(kernel_size=3, sigma=1.0):
3      k = kernel_size // 2
4      ax = np.linspace(-k, k, kernel_size)
5      xx, yy = np.meshgrid(ax, ax)
6      kernel = np.exp(-(xx**2 + yy**2) / (2 * sigma**2))
7      kernel = kernel / np.sum(kernel)
8      return kernel
9
10 # Filtro Gaussiano
11 def gaussian_filter(image, kernel_size=3, sigma=1.0):
12     kernel = gaussian_kernel(kernel_size, sigma)
13     filtered = convolve2d(image, kernel)
14     filtered = np.clip(filtered, 0, 255)
15     return filtered.astype(np.uint8)
16
```

Essas funções trabalham em conjunto para aplicar o **filtro Gaussiano** a uma imagem em tons de cinza. Esse tipo de filtro é amplamente utilizado para **suavização de imagens** e **remoção de ruído**, com a vantagem de preservar bordas de maneira mais eficiente do que métodos de média simples, graças à ponderação baseada na distribuição Gaussiana.

A função **gaussian_kernel** é responsável por gerar o kernel (máscara) Gaussiano, utilizado na convolução. Esse kernel é construído com base na função Gaussiana bidimensional, definida pela equação:

$$G(x, y) = 1/2\pi\sigma^2 \exp(-x^2 - y^2/2\sigma^2)$$

onde σ representa o desvio padrão da distribuição, controlando o grau de suavização, e **kernel_size** define a dimensão da matriz gerada (geralmente ímpar, como 3x3, 5x5 etc.). Para gerar o kernel, os pontos são organizados em um grid simétrico (via **meshgrid**), e os valores são calculados de acordo com a distância ao centro. Em seguida, a matriz é **normalizada** para que a soma total dos coeficientes seja 1, evitando alterações no brilho da imagem após a convolução.

A função **gaussian_filter** utiliza esse kernel como máscara convolucional, aplicando-o à imagem por meio da função **convolve2d**. A convolução pondera os pixels vizinhos de acordo com sua proximidade do centro, atribuindo maior peso aos valores mais próximos. Isso resulta em uma suavização que **reduz ruídos** como pequenas variações de intensidade, mantendo a integridade estrutural das regiões principais da imagem.

Após a convolução, a imagem resultante passa por uma etapa de **clipping**, restringindo os valores ao intervalo válido de [0, 255], e é convertida para o tipo **uint8**. O resultado é uma imagem suavizada, com menos ruídos e transições mais homogêneas, o que é fundamental no

pré-processamento de imagens médicas — especialmente quando se deseja remover imperfeições sem comprometer detalhes relevantes, como bordas celulares ou padrões patológicos sutis.

3.9. Filtro Mediana

```
1
2 # Filtro Mediana
3 def median_filter(image, kernel_size=3):
4     pad = kernel_size // 2
5     padded = np.pad(image, pad, mode='edge')
6     output = np.zeros_like(image)
7     for i in range(image.shape[0]):
8         for j in range(image.shape[1]):
9             window = padded[i:i+kernel_size, j:j+kernel_size]
10            output[i, j] = np.median(window)
11     return output
```

Essa função aplica o **filtro da mediana**, uma técnica de **suavização não linear** amplamente utilizada para **remoção de ruídos impulsivos**, especialmente o **ruído do tipo sal e pimenta**. Diferentemente dos filtros lineares (como o Gaussiano ou de média), que calculam combinações ponderadas de valores vizinhos, o filtro da mediana substitui cada pixel pelo valor **mediano** de uma vizinhança definida, preservando bordas e estruturas importantes da imagem.

O processo inicia-se com a definição do tamanho do kernel (janela), que deve ser ímpar (por padrão 3x3), e o cálculo do **padding** necessário para preservar o tamanho da imagem de saída. A imagem original é então expandida nas bordas utilizando o modo **'edge'**, que replica os valores da borda, evitando a criação de artefatos.

Para cada pixel da imagem, uma **janela deslizante** centrada sobre ele é extraída da imagem com **padding**. Dentro dessa janela, é calculado o valor **mediano**, ou seja, o valor central da lista de intensidades ordenadas. Esse valor é atribuído ao pixel correspondente da imagem de saída.

Ao contrário de filtros como o Gaussiano, que tendem a suavizar indiscriminadamente inclusive detalhes importantes, o filtro da mediana é particularmente eficaz para **preservar bordas** enquanto remove valores isolados e extremos; características típicas do ruído sal e pimenta.

5. Resultados obtidos com a implementação

5.1 Métricas utilizadas para análise entre os filtros

A fim de analisar quantitativamente o desempenho das técnicas de filtragem aplicadas, foram utilizadas cinco métricas amplamente reconhecidas no campo do processamento de imagens. Cada uma delas oferece uma perspectiva distinta sobre a fidelidade da imagem processada em relação à imagem original, permitindo uma avaliação abrangente da qualidade visual e estrutural.

MSE – Mean Squared Error (Erro Quadrático Médio):

O MSE mede o erro médio ao quadrado entre os valores de intensidade dos pixels da imagem original e da imagem processada. Trata-se de uma métrica direta e objetiva que quantifica a distorção introduzida pelo processamento, considerando o desvio absoluto em nível de pixel. Quanto menor o valor do MSE, maior a preservação da informação original.

RMSE – Root Mean Squared Error (Raiz do Erro Quadrático Médio):

O RMSE é simplesmente a raiz quadrada do MSE, o que o torna mais interpretável por estar na mesma escala dos valores de intensidade dos pixels (geralmente de 0 a 255). Ele fornece uma estimativa clara do erro médio por pixel e, assim como o MSE, quanto menor seu valor, melhor é a qualidade da imagem resultante.

PSNR – Peak Signal-to-Noise Ratio (Relação Pico de Sinal por Ruído):

O PSNR é uma métrica logarítmica que compara a potência máxima do sinal (imagem original) com a intensidade do ruído (diferença entre a imagem original e a processada). Por sua simplicidade e eficiência, é amplamente utilizada para medir a fidelidade de reconstruções de imagem. Em geral, valores acima de **30 dB** são considerados satisfatórios, indicando boa preservação dos detalhes.

SSIM – Structural Similarity Index (Índice de Similaridade Estrutural):

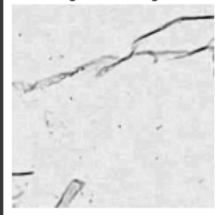
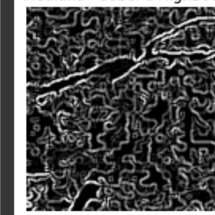
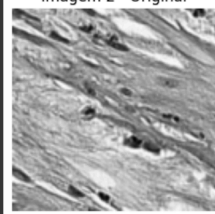
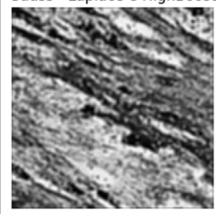
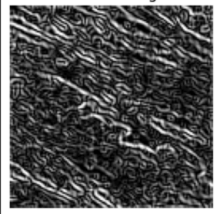
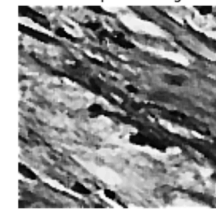
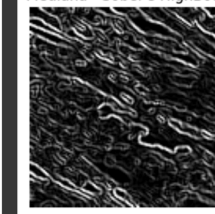
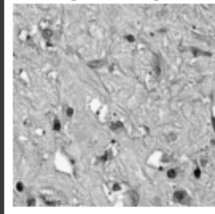
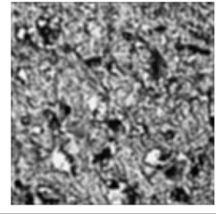
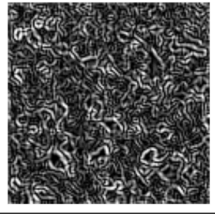
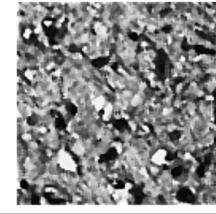
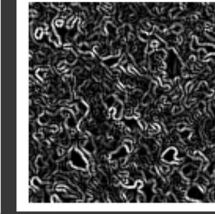
O SSIM é uma métrica perceptual que leva em consideração aspectos estruturais, de contraste e luminância, buscando simular a forma como o olho humano percebe diferenças visuais entre duas imagens. Seu valor varia de -1 a 1, sendo 1 a correspondência perfeita. Essa métrica é especialmente útil para detectar distorções que, embora pouco perceptíveis numericamente (como em MSE ou PSNR), afetam a estrutura visual percebida.

GMSD – Gradient Magnitude Similarity Deviation:

O GMSD avalia a similaridade entre imagens com base na diferença entre as magnitudes dos gradientes, isto é, nas bordas e transições de intensidade — elementos visuais críticos para a percepção de forma e estrutura. Valores mais baixos de GMSD indicam maior preservação de contornos e estruturas da imagem original, sendo, portanto, uma métrica sensível à qualidade estrutural das imagens após a filtragem.

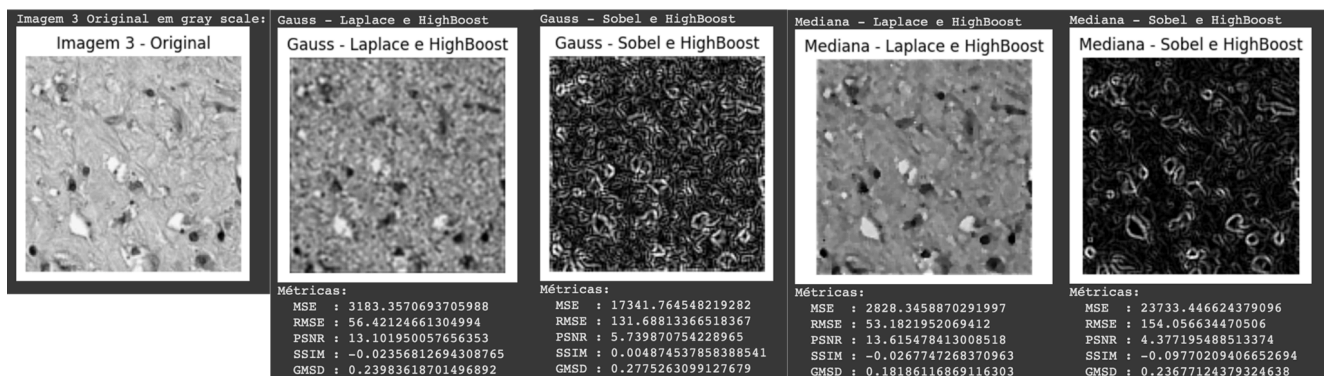
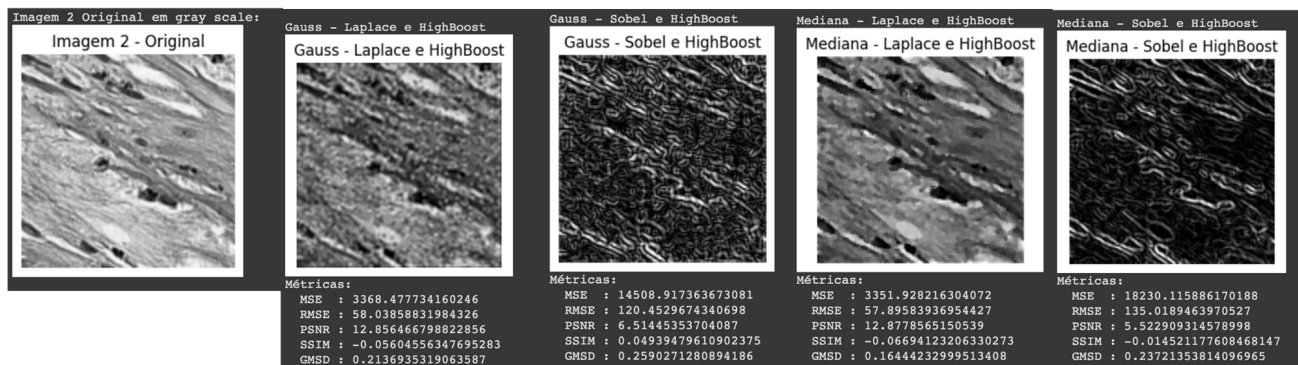
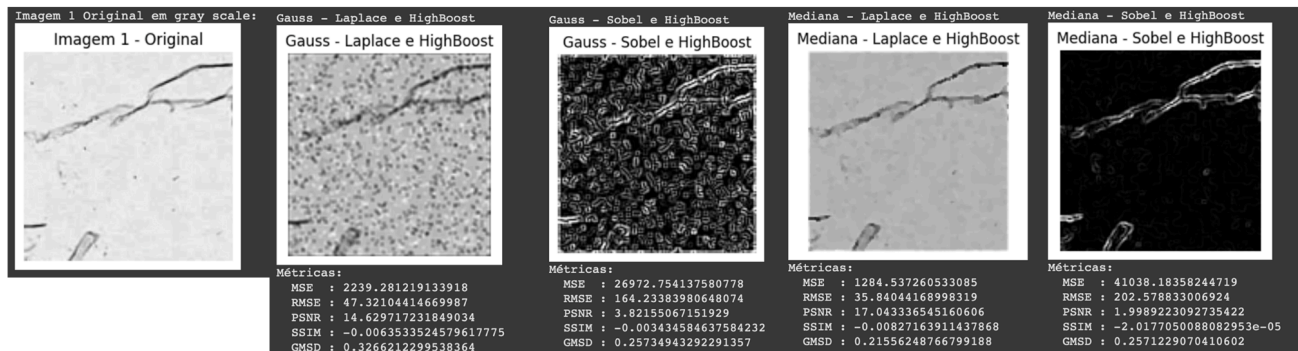
5.2. Resultados

5.2.1. Equalização de Histograma

Imagem 1 Original: Imagem 1 - Original 	Gauss - Laplace e HighBoost  Métricas: MSE : 7751.991049198103 RMSE : 88.04539198162561 PSNR : 9.236670982810987 SSIM : -0.011106127603899578 GMSD : 0.2143205314810502	Gauss - Sobel e HighBoost  Métricas: MSE : 21848.67911783854 RMSE : 147.81298697285885 PSNR : 4.736551744435723 SSIM : -0.005606539465950702 GMSD : 0.2162784831592657	Mediana - Laplace e HighBoost  Métricas: MSE : 9504.014353500177 RMSE : 97.48853447200946 PSNR : 8.351732773341995 SSIM : -0.014307370963876723 GMSD : 0.23241927775190913	Mediana - Sobel e HighBoost  Métricas: MSE : 27127.549675835504 RMSE : 164.70443125743614 PSNR : 3.796697934359438 SSIM : -0.006837835847321925 GMSD : 0.23402979887878225
Imagem 2 Original: Imagem 2 - Original 	Gauss - Laplace e HighBoost  Métricas: MSE : 1953.5964647985384 RMSE : 44.1995075176018 PSNR : 15.222455001597508 SSIM : -0.08924224053468774 GMSD : 0.2123951728679496	Gauss - Sobel e HighBoost  Métricas: MSE : 13451.93544665678 RMSE : 115.98247905031509 PSNR : 6.8429558631898 SSIM : -0.005571950492009106 GMSD : 0.26640145571773105	Mediana - Laplace e HighBoost  Métricas: MSE : 2611.9182742818193 RMSE : 51.106929806845365 PSNR : 13.961207769263249 SSIM : -0.1549311403558274 GMSD : 0.1950161911495128	Mediana - Sobel e HighBoost  Métricas: MSE : 16570.083779417437 RMSE : 128.72483746121972 PSNR : 5.93755656621977 SSIM : -0.04752786900010894 GMSD : 0.2531132398647645
Imagem 3 Original: Imagem 3 - Original 	Gauss - Laplace e HighBoost  Métricas: MSE : 3691.81637968652 RMSE : 60.76031912100627 PSNR : 12.45840268533545 SSIM : -0.04130188455259187 GMSD : 0.23353051561370727	Gauss - Sobel e HighBoost  Métricas: MSE : 15213.772989155332 RMSE : 123.34412425873934 PSNR : 6.308434291837407 SSIM : -0.009373692779513932 GMSD : 0.2768795765122774	Mediana - Laplace e HighBoost  Métricas: MSE : 5058.704350966465 RMSE : 71.12456924977799 PSNR : 11.090410624626667 SSIM : -0.05275072229549844 GMSD : 0.2410120318008408	Mediana - Sobel e HighBoost  Métricas: MSE : 18045.393897539303 RMSE : 134.33314519335613 PSNR : 5.567139945244377 SSIM : -0.012482075049031064 GMSD : 0.2723137810398397

5.2.2. Correção de Gamma

5.2.2.1. Gamma 0.5



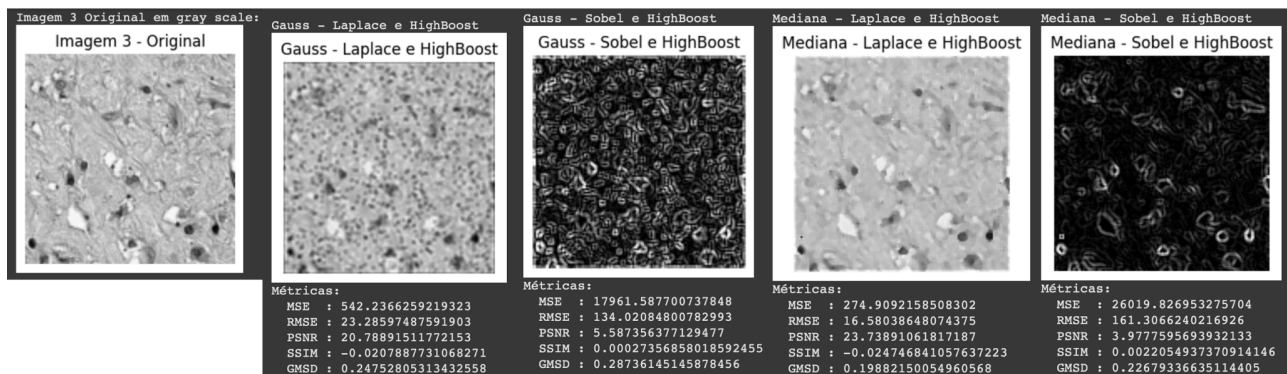
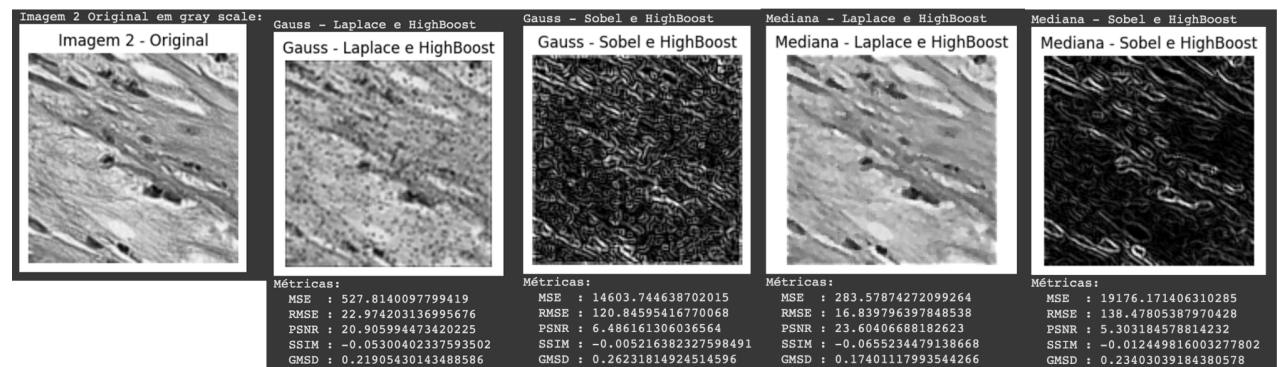
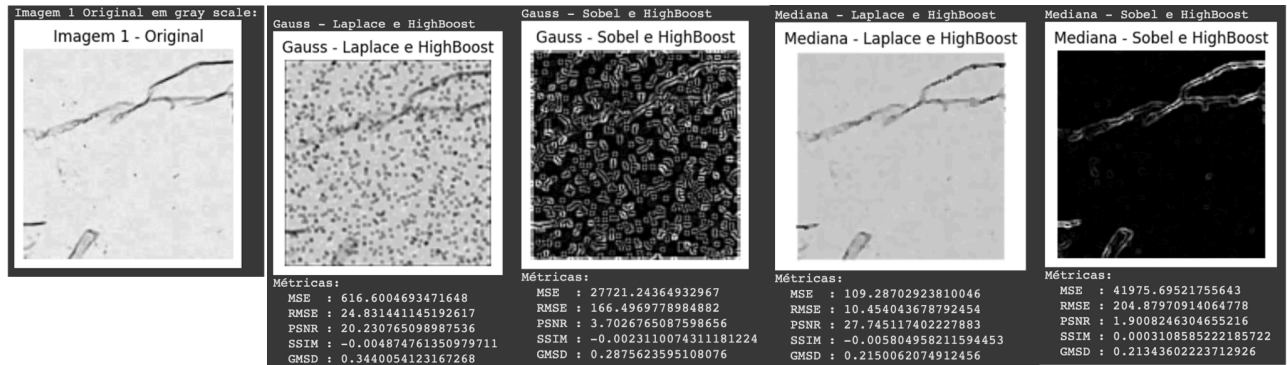
5.2.2.2. Gamma 0.9

Imagem 1 Original em gray scale:	Gauss - Laplace e HighBoost	Gauss - Sobel e HighBoost	Mediana - Laplace e HighBoost	Mediana - Sobel e HighBoost
Métricas: MSE : 728.4528395250105 RMSE : 26.98986549660836 PSNR : 19.50678920337785 SSIM : -0.0053695484095017815 GMSD : 0.3407980158666003	Métricas: MSE : 27490.523869267217 RMSE : 165.80266544681126 PSNR : 3.7389734487441677 SSIM : -0.0026884143498328255 GMSD : 0.28011566007051314	Métricas: MSE : 104.03717721256203 RMSE : 10.199861627128184 PSNR : 27.95891800673938 SSIM : -0.006537437054273621 GMSD : 0.21679915089512325	Métricas: MSE : 41726.29841368875 RMSE : 204.27016036046174 PSNR : 1.9267050119363716 SSIM : 0.00025802155414076993 GMSD : 0.2275252747357414	

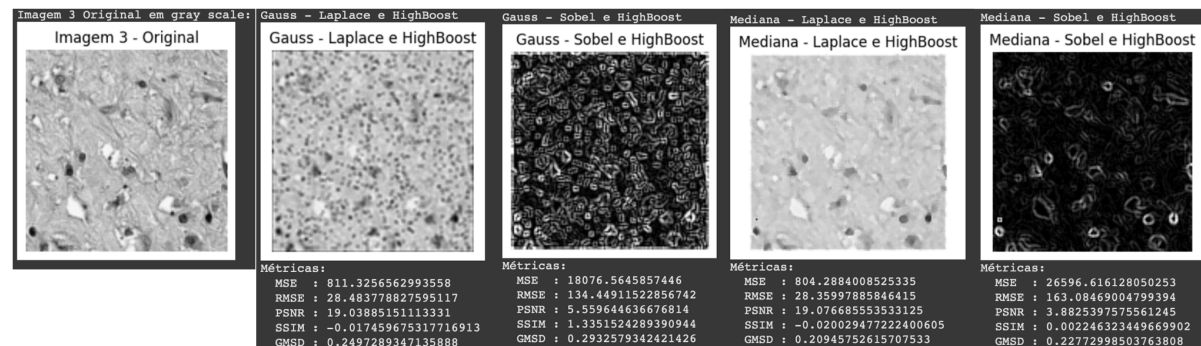
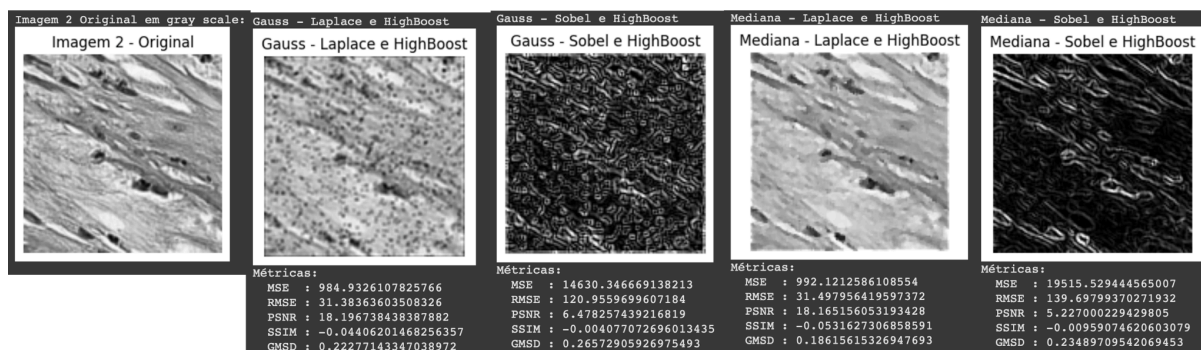
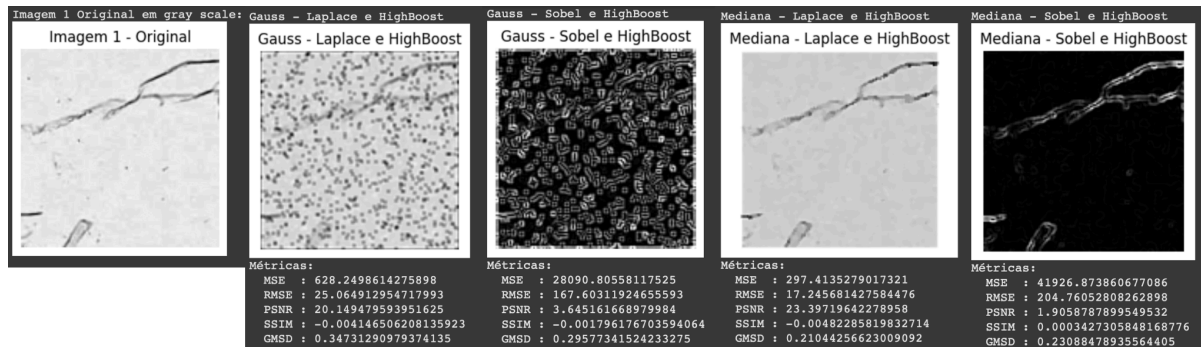
Imagem 2 Original em gray scale:	Gauss - Laplace e HighBoost	Gauss - Sobel e HighBoost	Mediana - Laplace e HighBoost	Mediana - Sobel e HighBoost
Métricas: MSE : 610.021278111964 RMSE : 24.698608829486002 PSNR : 20.27735376995762 SSIM : -0.05759829835073374 GMSD : 0.2166577197854787	Métricas: MSE : 14658.055469654224 RMSE : 121.07045663436735 PSNR : 6.470040000315586 SSIM : -0.00911886227910532 GMSD : 0.26054139546813176	Métricas: MSE : 278.8167421976724 RMSE : 16.697806508570892 PSNR : 23.67761512439943 SSIM : -0.07184329550287302 GMSD : 0.16756842072054864	Métricas: MSE : 18933.159543637878 RMSE : 137.59781809185012 PSNR : 5.358572662983224 SSIM : -0.010284454355531443 GMSD : 0.23512553642540196	

Imagem 3 Original em gray scale:	Gauss - Laplace e HighBoost	Gauss - Sobel e HighBoost	Mediana - Laplace e HighBoost	Mediana - Sobel e HighBoost
Métricas: MSE : 648.2833189316736 RMSE : 25.46140842395946 PSNR : 20.013155140130095 SSIM : -0.022641067902350497 GMSD : 0.24591355994572528	Métricas: MSE : 18026.575809431662 RMSE : 134.2630843137147 PSNR : 5.571671215719985 SSIM : 0.0004578185410979434 GMSD : 0.2830551500612049	Métricas: MSE : 246.10738141095186 RMSE : 15.687809962227101 PSNR : 24.219557213302 SSIM : -0.027213147200442385 GMSD : 0.19160891359517107	Métricas: MSE : 25571.23033688392 RMSE : 159.9100695293574 PSNR : 4.053287366481846 SSIM : 0.001960656942147793 GMSD : 0.22724719979665306	

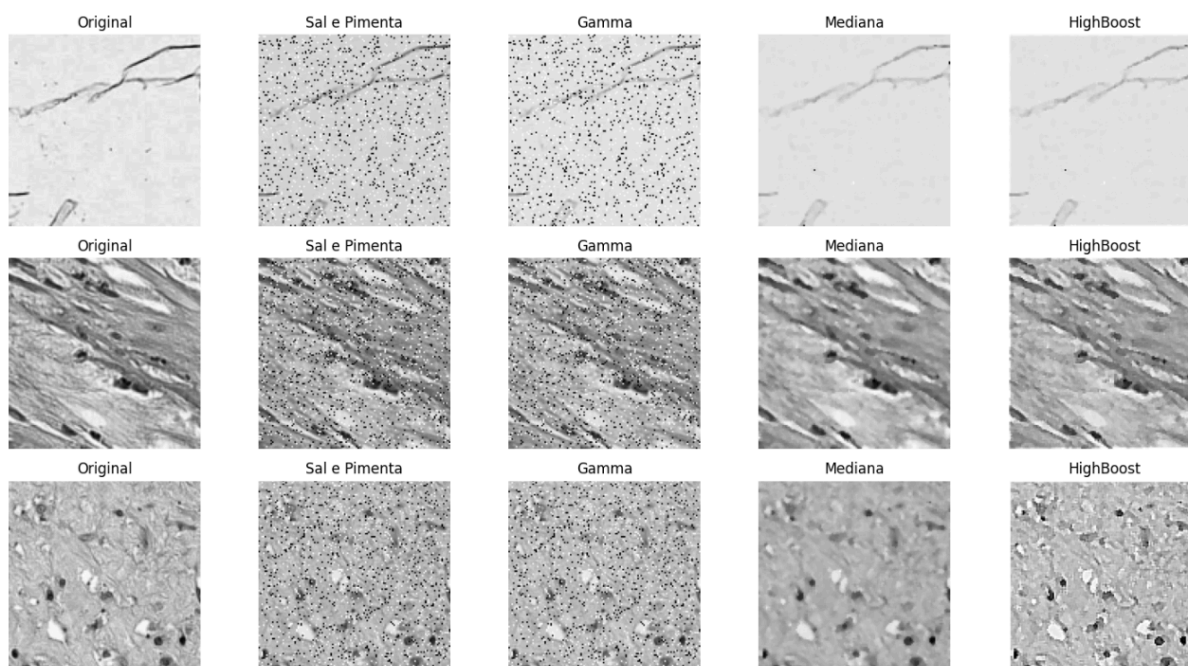
5.2.2.3. Gamma 1.1



5.2.2.4. Gamma 1.5



5.3 Melhores resultados obtidos



Durante a execução do pipeline de processamento, foram aplicadas diferentes combinações de parâmetros às três imagens selecionadas do conjunto **PathMNIST**, visando encontrar um equilíbrio entre a remoção eficaz de ruído e a preservação das estruturas anatômicas relevantes. A seleção dos parâmetros foi realizada com base na análise visual das imagens processadas e nos valores das métricas quantitativas (MSE, RMSE, PSNR, SSIM e GMSD), buscando maximizar a qualidade visual e estrutural da imagem final.

Os melhores resultados foram obtidos com os seguintes valores para os parâmetros, aplicados individualmente às imagens 1, 2 e 3, respectivamente:

- **Correção Gama (gamma):**

- Imagem 1: 1.5
- Imagens 2 e 3: 1.1

Esses valores foram escolhidos para ajustar o brilho e o contraste de acordo com as características particulares de cada imagem. Um valor de gama mais alto em 1.5 foi necessário para clarear a imagem 1, que apresentava regiões mais escuras, enquanto 1.1 foi suficiente para destacar detalhes nas demais imagens sem superexpor áreas mais claras.

- **Tamanho do Kernel para Filtragem (mediana/gaussiana):**

- Todas as imagens: 3x3

Esse tamanho de kernel mostrou-se eficaz para remover o ruído sal e pimenta introduzido artificialmente, ao mesmo tempo em que preservava bordas e detalhes importantes. Kernels maiores não apresentaram ganhos significativos e, em alguns casos, causaram perda de definição estrutural.

- **Fator de Realce no Filtro High-Boost (A):**

- Imagem 1: 1.25
- Imagem 2: 2.0
- Imagem 3: 5.0

O valor de A foi ajustado com base no nível de detalhes desejado em cada imagem. Para a imagem 1, um realce sutil foi suficiente para acentuar as bordas. Já nas imagens 2 e 3, valores progressivamente maiores foram utilizados para reforçar ainda mais as estruturas internas e contornos, especialmente em regiões com contraste naturalmente mais baixo.

Essa seleção de parâmetros demonstrou bom desempenho tanto na **redução de ruído**, quanto no **realce de bordas e contraste**, resultando em imagens visualmente mais nítidas e adequadas para interpretações clínicas ou análises computacionais subsequentes.

6. Análise e Discussão

A análise comparativa entre os diferentes fluxos de processamento — com e sem aplicação da **correção gama** — revela insights importantes sobre o desempenho das combinações de filtros no aprimoramento das imagens médicas do dataset **PathMNIST**. Para isso, foram utilizadas cinco métricas: **MSE**, **RMSE**, **PSNR**, **SSIM** e **GMSD**.

Efeito da Correção Gama

A introdução da **correção gama** antes das etapas de filtragem e realce resultou em uma melhora significativa nos valores das métricas, principalmente para as combinações envolvendo **filtro de mediana + Laplaciano + High-Boost**. Por exemplo, na **Imagem 1**, a MSE caiu de **9504.01** (sem gama) para **297.41** (com gama), e o PSNR subiu de **8.35 dB** para **23.39 dB**, indicando uma redução expressiva da distorção com melhora no contraste e clareza visual. A **Imagem 2** seguiu tendência semelhante, com PSNR de **13.96 dB** (sem gama) subindo para **18.16 dB** (com gama).

Além disso, o **GMSD**, que avalia a preservação de estruturas e bordas, apresentou **valores mais baixos com gama**; sinalizando uma imagem visualmente mais próxima da original. A diferença foi clara na Imagem 2: **GMSD = 0.195 (sem gama) versus 0.186 (com gama)**. Esses resultados confirmam que a correção gama contribui significativamente para preparar a imagem, realçando regiões de baixo contraste e melhorando o desempenho subsequente dos filtros.

Comparação Entre Laplaciano e Sobel

Tanto com quanto sem correção gama, as combinações que utilizam o **filtro Laplaciano** superaram as versões que utilizaram o **filtro de Sobel**, especialmente nas métricas **MSE**, **PSNR** e **GMSD**. O filtro de Sobel, por sua natureza direcional, não se mostrou tão eficaz em capturar os contornos complexos e não orientados das imagens histopatológicas. Na **Imagem 3 com gama**, por exemplo, o método **Mediana + Laplace + High-Boost** apresentou **GMSD de 0.209**, comparado a **0.227** do método com Sobel; reforçando sua superioridade em preservar bordas.

Observações do SSIM

Embora os valores de **SSIM** tenham permanecido em faixas negativas ou próximas de zero na maioria dos casos (o que pode ocorrer devido à natureza intensa das transformações e compressão da faixa dinâmica), o comportamento relativo da métrica acompanhou os demais indicadores. Os melhores SSIMs foram observados nas imagens com gama e filtragem mediana, sugerindo uma maior fidelidade estrutural ao original mesmo com as limitações do SSIM em imagens com alta compactação de tonalidades e resolução reduzida.

7. Conclusão

Com base nas análises visuais e nas métricas quantitativas, a sequência de processamento composta por **Correção Gama** → **Filtro de Mediana** → **High-Boost Filtering** apresentou os melhores resultados no tratamento das imagens histopatológicas do conjunto **PathMNIST**.

A **correção gama** foi fundamental para melhorar a distribuição tonal, principalmente em imagens com regiões subexpostas. O **filtro de mediana** provou ser o mais eficaz na remoção de ruído sal e pimenta, enquanto o realce dado por **High-Boost**, destacou bordas e estruturas sem comprometer a integridade visual.

As métricas **MSE**, **RMSE** e **PSNR** indicaram menor distorção e maior fidelidade em relação às imagens originais, enquanto o **GMSD** confirmou a preservação de contornos importantes. Com isso, conclui-se que o fluxo com gama e mediana é o mais adequado para esse tipo de imagem médica, proporcionando uma saída mais limpa, nítida e informativamente rica; essencial em contextos clínicos e diagnósticos assistidos por computador.

8. Referências

YAN, K.; WANG, X.; ZHOU, Y. *MedMNIST v2: A large-scale lightweight benchmark for 2D and 3D biomedical image classification*. Scientific Data, 2023. Disponível em: <https://medmnist.com>. Acesso em: 14 abr. 2025.

SANTOS, M. *Salt and Pepper Noise in Python*. Disponível em: <https://www.geeksforgeeks.org/python-introduction-to-salt-and-pepper-noise/>. Acesso em: 14 abr. 2025.

CODEX. *Python #10 - Laplacian Filter in Spatial Domain using Python*. YouTube, 2021. Disponível em: <https://www.youtube.com/watch?v=wEsaWpuYb70>. Acesso em: 14 abr. 2025.

CODEX. *Python #13 - Edge Detection using Sobel Operator in Python*. YouTube, 2021. Disponível em: <https://www.youtube.com/watch?v=nW7g3zT5NHA>. Acesso em: 14 abr. 2025.

CODEX. *Python #11 - Unsharp Masking and High Boost Filtering in Spatial Domain*. YouTube, 2021. Disponível em: <https://www.youtube.com/watch?v=pkz8CNnt-jw>. Acesso em: 14 abr. 2025.

METEHANC. *Python-Median-Filter: Simple implementation of median filter in python to remove noise from the images*. GitHub, 2020. Disponível em: <https://github.com/MeteHanC/Python-Median-Filter>. Acesso em: 14 abr. 2025.

ADAMIAO. *Sobel Filter Tutorial: A python code showing how to apply a Sobel filter for edge detection*. GitHub, 2020. Disponível em: <https://github.com/adamiao/sobel-filter-tutorial>. Acesso em: 14 abr. 2025.